



Informatik II – Machine Learning

Informatik II – Woche 1

12. / 13. Mai 2026

Heutiges Programm

Entscheidungsbäume

Programmierübungen

k-Nearest Neighbors

Lineare Regression

Programmierübung

1. Entscheidungsbäume

Motivation: ein Flussdiagramm aus Ja/Nein-Fragen

Die Idee in einem Satz:

*Eine Folge von Ja/Nein-Fragen über die **Merkmale** stellen und den Antworten zur Vorhersage folgen.*

Motivation: ein Flussdiagramm aus Ja/Nein-Fragen

Die Idee in einem Satz:

*Eine Folge von Ja/Nein-Fragen über die **Merkmale** stellen und den Antworten zur Vorhersage folgen.*

Das ist das ganze Modell, ein **Flussdiagramm**, von oben nach unten gelesen.

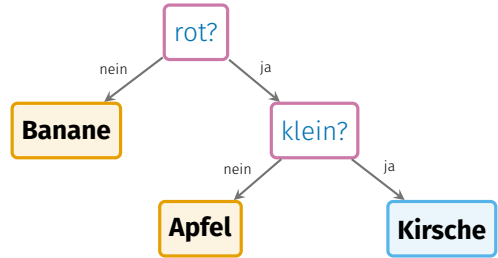
Motivation: ein Flussdiagramm aus Ja/Nein-Fragen

Die Idee in einem Satz:

*Eine Folge von Ja/Nein-Fragen über die **Merkmale** stellen und den Antworten zur Vorhersage folgen.*

Das ist das ganze Modell, ein **Flussdiagramm**, von oben nach unten gelesen.

- Keine Gleichungen, keine Parameter zum Anpassen.
- Das Modell ist das Flussdiagramm selbst.

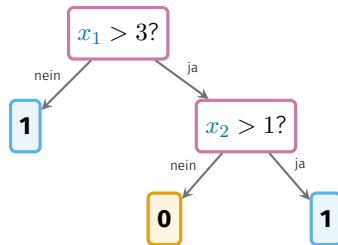


Zwei Fragen klassifizieren die Frucht.

Aufbau eines Entscheidungsbaums

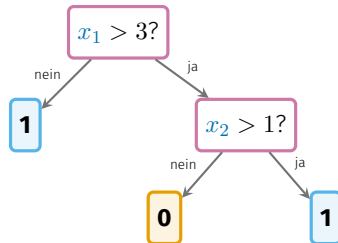
■ Innerer Knoten (violett):

Ja/Nein-Frage über *ein* Merkmal, z. B.
„ $x_j > t_j$ “. Der oberste ist die **Wurzel**.



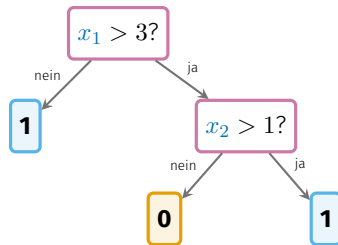
Aufbau eines Entscheidungsbaums

- **Innerer Knoten** (violett):
Ja/Nein-Frage über *ein* Merkmal, z. B.
„ $x_j > t_j$ “. Der oberste ist die **Wurzel**.
- **Blatt** (orange oder blau, eine Farbe pro Klasse): eine Vorhersage $\hat{y}$ am unteren Ende des Baums.



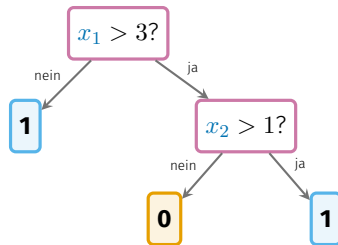
Aufbau eines Entscheidungsbaums

- **Innerer Knoten** (violett):
Ja/Nein-Frage über *ein* Merkmal, z. B. „ $x_j > t_j$ “. Der oberste ist die **Wurzel**.
- **Blatt** (orange oder blau, eine Farbe pro Klasse): eine Vorhersage $\hat{y}$ am unteren Ende des Baums.
- **Tiefe**: längster Pfad von der Wurzel zu einem Blatt, also die maximale Anzahl Fragen pro Datenpunkt.

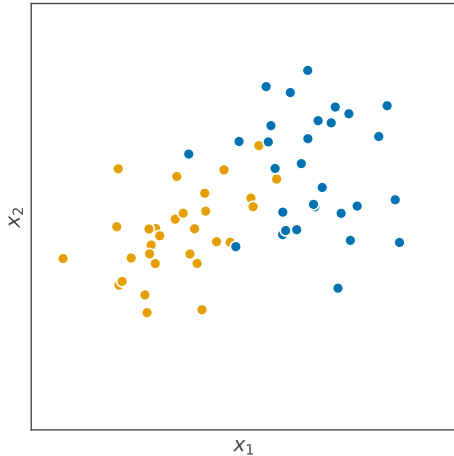


Aufbau eines Entscheidungsbaums

- **Innerer Knoten** (violett):
Ja/Nein-Frage über *ein* Merkmal, z. B. „ $x_j > t_j'$ “. Der oberste ist die **Wurzel**.
- **Blatt** (orange oder blau, eine Farbe pro Klasse): eine Vorhersage $\hat{y}$ am unteren Ende des Baums.
- **Tiefe**: längster Pfad von der Wurzel zu einem Blatt, also die maximale Anzahl Fragen pro Datenpunkt.
- **Vorhersage**: von der Wurzel hinab den Antworten folgen, bis ein Blatt erreicht ist. Den dort gespeicherten Wert ausgeben.

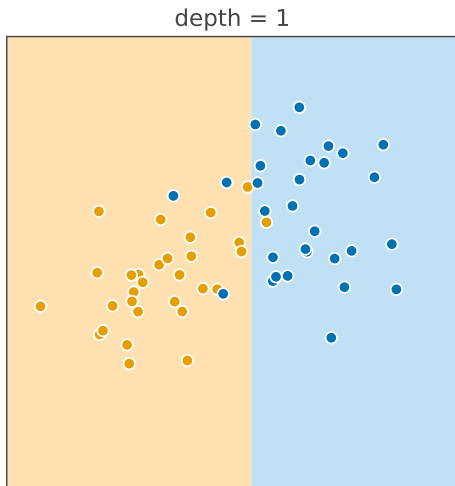


Entscheidungsregionen: achsenparallele Rechtecke



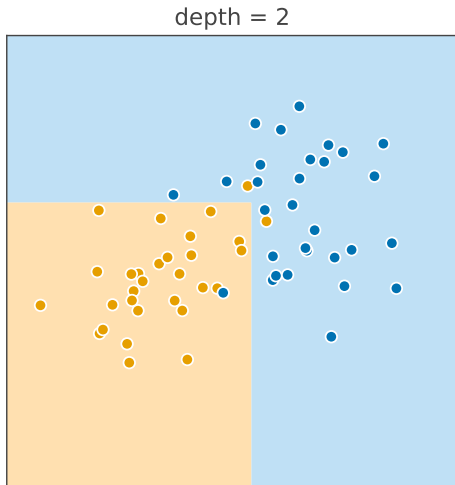
■ Zwei Klassen (A, B) in 2D.

Entscheidungsregionen: achsenparallele Rechtecke



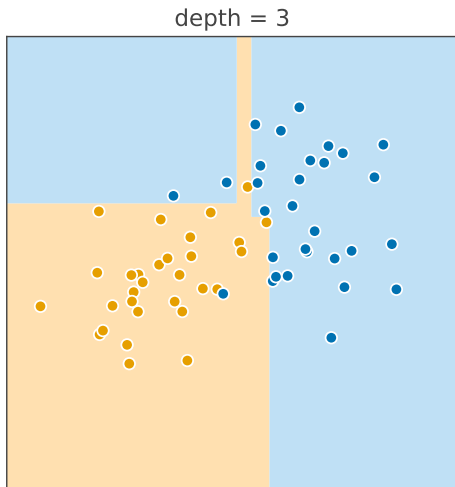
- Zwei Klassen (A, B) in 2D.
- **Tiefe 1:** ein Schnitt $\Rightarrow$ zwei Regionen.

Entscheidungsregionen: achsenparallele Rechtecke



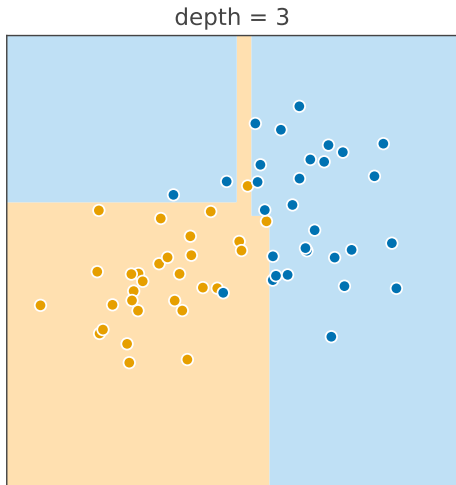
- Zwei Klassen (A, B) in 2D.
- **Tiefe 1:** ein Schnitt $\Rightarrow$ zwei Regionen.
- **Tiefe 2:** jede Region erneut teilbar $\Rightarrow$ bis zu vier.

Entscheidungsregionen: achsenparallele Rechtecke



- Zwei Klassen (A, B) in 2D.
- **Tiefe 1:** ein Schnitt $\Rightarrow$ zwei Regionen.
- **Tiefe 2:** jede Region erneut teilbar $\Rightarrow$ bis zu vier.
- **Tiefe 3:** bis zu acht Rechtecke.

Entscheidungsregionen: achsenparallele Rechtecke



- Zwei Klassen (A, B) in 2D.
- **Tiefe 1:** ein Schnitt $\Rightarrow$ zwei Regionen.
- **Tiefe 2:** jede Region erneut teilbar $\Rightarrow$ bis zu vier.
- **Tiefe 3:** bis zu acht Rechtecke.
- Zwei gleichwertige Sichten:
 - Flussdiagramm aus Ja/Nein-Fragen;
 - Zerlegung des **Merkmalsraums** (ein Rechteck pro Blatt).

Baum aufbauen: gierig, von oben

Der Baum wird *rekursiv* aufgebaut, beginnend bei der Wurzel mit **allen** Trainingsdaten:

Baum aufbauen: gierig, von oben

Der Baum wird *rekursiv* aufgebaut, beginnend bei der Wurzel mit **allen** Trainingsdaten:

1. Unter allen Kandidaten-Aufteilungen „ $x_j > t_j$ “ diejenige wählen, die die Klassen im aktuellen Knoten am besten trennt.

Baum aufbauen: gierig, von oben

Der Baum wird *rekursiv* aufgebaut, beginnend bei der Wurzel mit **allen** Trainingsdaten:

1. Unter allen Kandidaten-Aufteilungen „ $x_j > t_j$ “ diejenige wählen, die die Klassen im aktuellen Knoten am besten trennt.
2. Jeden Datenpunkt gemäss seiner Antwort an das linke oder rechte Kind weiterleiten.

Baum aufbauen: gierig, von oben

Der Baum wird *rekursiv* aufgebaut, beginnend bei der Wurzel mit **allen** Trainingsdaten:

1. Unter allen Kandidaten-Aufteilungen „ $x_j > t_j'$ “ diejenige wählen, die die **Klassen im aktuellen Knoten am besten trennt**.
2. Jeden Datenpunkt gemäss seiner Antwort an das **linke** oder **rechte** Kind weiterleiten.
3. In jedem Kind wiederholen, bis eine **Abbruchregel** erfüllt ist.

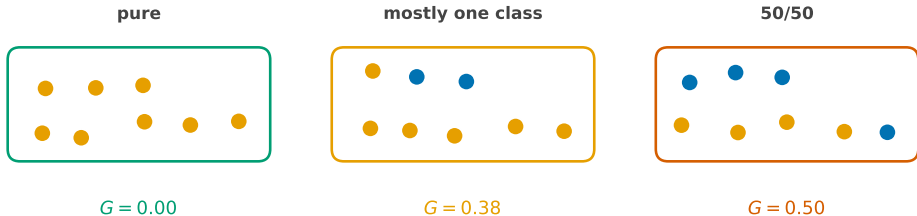
Baum aufbauen: gierig, von oben

Der Baum wird *rekursiv* aufgebaut, beginnend bei der Wurzel mit **allen** Trainingsdaten:

1. Unter allen Kandidaten-Aufteilungen „ $x_j > t_j$ “ diejenige wählen, die die **Klassen im aktuellen Knoten am besten trennt**.
2. Jeden Datenpunkt gemäss seiner Antwort an das **linke** oder **rechte** Kind weiterleiten.
3. In jedem Kind wiederholen, bis eine **Abbruchregel** erfüllt ist.

„Am besten“ muss präzisiert werden: man braucht ein numerisches **Aufteilungskriterium**, das misst, wie *gemischt* ein Knoten ist.

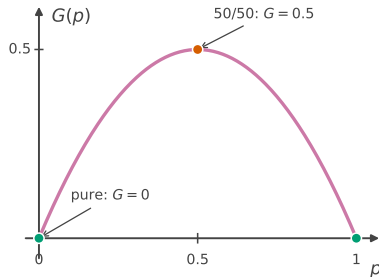
Unreinheit: “wie gemischt ist die Gruppe?”



Eine gute Aufteilung schickt (überwiegend) **eine** Klasse nach links und (überwiegend) **die andere** Klasse nach rechts, sodass die Kinder Richtung **reiner** Gruppen gedrängt werden.

Splitkriterium: der Gini-Index

- p_c = Anteil der Klasse c im Knoten.



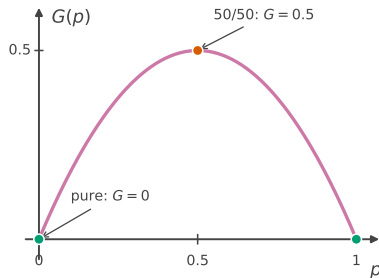
$$G = 2p(1-p)$$

p = Anteil positiver Klasse

Splitkriterium: der Gini-Index

- p_c = Anteil der Klasse c im Knoten.
- **Gini-Unreinheit** (K Klassen):

$$G = 1 - \sum_c p_c^2.$$



$$G = 2p(1-p)$$

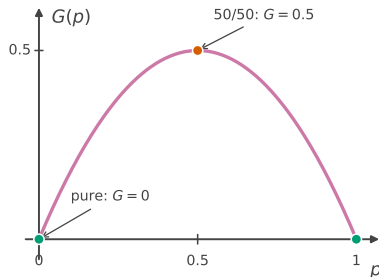
p = Anteil positiver Klasse

Splitkriterium: der Gini-Index

- p_c = Anteil der Klasse c im Knoten.
- **Gini-Unreinheit** (K Klassen):

$$G = 1 - \sum_c p_c^2.$$

- Zwei Klassen: $G = 2p(1-p)$.
Rein: $G = 0$. 50/50: $G = \frac{1}{2}$ (max.).



$$G = 2p(1-p)$$

p = Anteil positiver Klasse

Splitkriterium: der Gini-Index

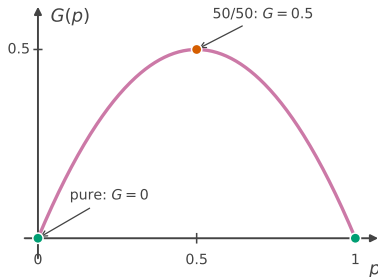
- p_c = Anteil der Klasse c im Knoten.
- **Gini-Unreinheit** (K Klassen):

$$G = 1 - \sum_c p_c^2.$$

- Zwei Klassen: $G = 2p(1-p)$.
Rein: $G = 0$. 50/50: $G = \frac{1}{2}$ (max.).
- **Split-Score** (gewichtet):

$$G_{\text{split}} = \frac{n_L}{n} G_L + \frac{n_R}{n} G_R.$$

Aufteilung mit **kleinstem** G_{split} wählen.



$$G = 2p(1-p)$$

p = Anteil positiver Klasse

Wann aufhören? Die Tiefe als zentraler Schalter

Abbruchregeln, also wann ein Knoten zum Blatt erklärt wird:

- eine gewählte **maximale Tiefe** ist erreicht;
- der Knoten enthält weniger als m Datenpunkte;
- der Knoten ist (fast) rein (G unter einem Schwellenwert).

Wann aufhören? Die Tiefe als zentraler Schalter

Abbruchregeln, also wann ein Knoten zum Blatt erklärt wird:

- eine gewählte **maximale Tiefe** ist erreicht;
- der Knoten enthält weniger als m Datenpunkte;
- der Knoten ist (fast) rein (G unter einem Schwellenwert).

Die **maximale Tiefe** ist der wichtigste Schalter (ein **Hyperparameter**, siehe Training-Lektion später). Sie steuert, wie viele Fragen der Baum stellen darf, und damit die Komplexität der Entscheidungsgrenze.

Wann aufhören? Die Tiefe als zentraler Schalter

Abbruchregeln, also wann ein Knoten zum Blatt erklärt wird:

- eine gewählte **maximale Tiefe** ist erreicht;
- der Knoten enthält weniger als m Datenpunkte;
- der Knoten ist (fast) rein (G unter einem Schwellenwert).

Die **maximale Tiefe** ist der wichtigste Schalter (ein **Hyperparameter**, siehe Training-Lektion später). Sie steuert, wie viele Fragen der Baum stellen darf, und damit die Komplexität der Entscheidungsgrenze.

- Flacher Baum $\Rightarrow$ zu wenige Fragen $\Rightarrow$ *Unteranpassung* (echte Struktur verfehlt).

Wann aufhören? Die Tiefe als zentraler Schalter

Abbruchregeln, also wann ein Knoten zum Blatt erklärt wird:

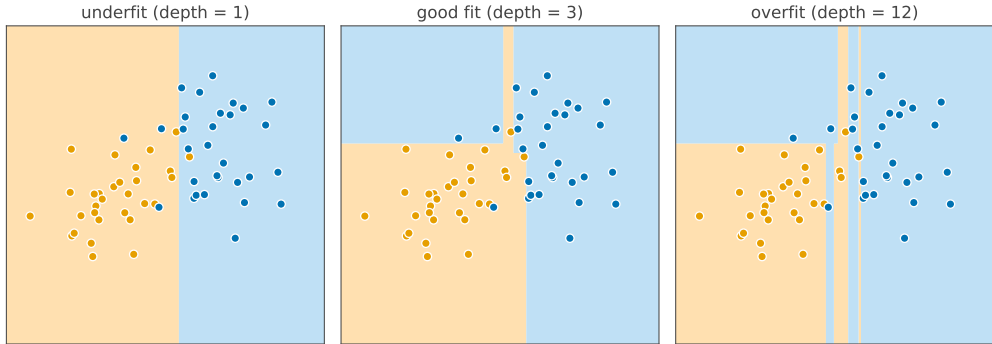
- eine gewählte **maximale Tiefe** ist erreicht;
- der Knoten enthält weniger als m Datenpunkte;
- der Knoten ist (fast) rein (G unter einem Schwellenwert).

Die **maximale Tiefe** ist der wichtigste Schalter (ein **Hyperparameter**, siehe Training-Lektion später). Sie steuert, wie viele Fragen der Baum stellen darf, und damit die Komplexität der Entscheidungsgrenze.

- Flacher Baum $\Rightarrow$ zu wenige Fragen $\Rightarrow$ *Unteranpassung* (echte Struktur verfehlt).
- Sehr tiefer Baum $\Rightarrow$ jedes Blatt enthält einen einzelnen Trainingspunkt $\Rightarrow$ 100% auf Trainingsdaten, schlecht auf neuen Daten $\Rightarrow$ *Überanpassung* (Auswendiglernen).

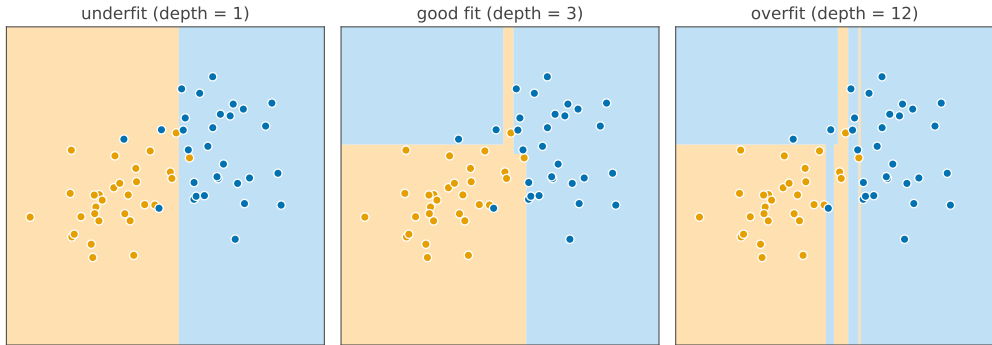
Über- und Unteranpassung folgen formell in der Training-Lektion.

Tiefe visuell: zu flach, gut, zu tief



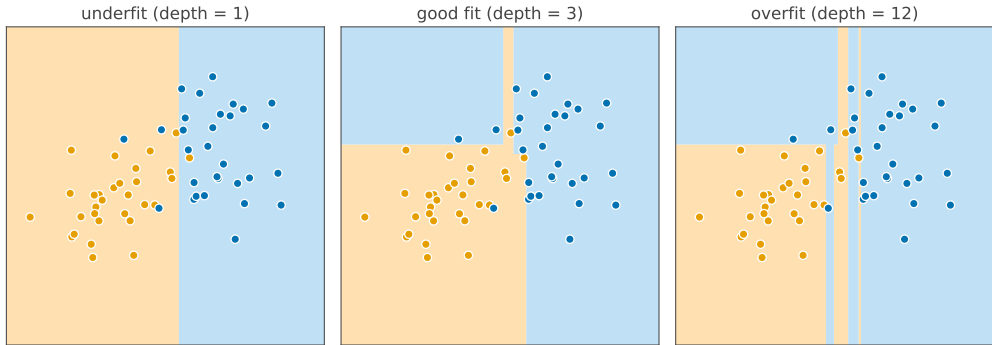
- Links: zu **flach**, um die Struktur zu erfassen.

Tiefe visuell: zu flach, gut, zu tief



- Links: zu **flach**, um die Struktur zu erfassen.
- Mitte: mittlere Tiefe, der **optimale Bereich**.

Tiefe visuell: zu flach, gut, zu tief



- Links: zu **flach**, um die Struktur zu erfassen.
- Mitte: mittlere Tiefe, der **optimale Bereich**.
- Rechts: tief, winzige Rechtecke um einzelne Punkte, also **Auswendiglernen**.

Massnahmen gegen Überanpassung

Drei Stellschrauben (Hyperparameter, wie wir in der Training-Lektion sehen werden), welche die Kapazität des Baums begrenzen:

- **max_depth**: die Tiefe des Baums deckeln.

Massnahmen gegen Überanpassung

Drei Stellschrauben (Hyperparameter, wie wir in der Training-Lektion sehen werden), welche die Kapazität des Baums begrenzen:

- **max_depth**: die Tiefe des Baums deckeln.
- **min_samples_leaf**: jedes Blatt muss mindestens m Trainingspunkte enthalten, sodass keine Blätter um einzelne Ausreisser entstehen.

Alle drei Schalter begrenzen, wie flexibel sich der Baum an die Trainingsdaten anpassen kann: ein weniger flexibler Baum verfehlt unter Umständen echte Struktur, ein sehr flexibler Baum lernt die Trainingspunkte auswendig und sagt auf neuen Daten schlecht voraus.

Massnahmen gegen Überanpassung

Drei Stellschrauben (Hyperparameter, wie wir in der Training-Lektion sehen werden), welche die Kapazität des Baums begrenzen:

- **max_depth**: die Tiefe des Baums deckeln.
- **min_samples_leaf**: jedes Blatt muss mindestens m Trainingspunkte enthalten, sodass keine Blätter um einzelne Ausreisser entstehen.
- **Pruning**: einen tiefen Baum wachsen lassen und anschliessend Zweige *beschneiden*, die auf einer zurückgehaltenen Menge nichts bringen.
(Nur als Idee; in dieser Vorlesung nicht implementiert.)

Alle drei Schalter begrenzen, wie flexibel sich der Baum an die Trainingsdaten anpassen kann: ein weniger flexibler Baum verfehlt unter Umständen echte Struktur, ein sehr flexibler Baum lernt die Trainingspunkte auswendig und sagt auf neuen Daten schlecht voraus.

Kategorische Merkmale

Echte Datensätze enthalten oft nicht-numerische Merkmale: *Land, Farbe, Blutgruppe*.

Kategorische Merkmale

Echte Datensätze enthalten oft nicht-numerische Merkmale: *Land*, *Farbe*, *Blutgruppe*.

Ein Baum verarbeitet sie *direkt*, denn eine Frage kann einfach nach der Kategoriezugehörigkeit fragen:

- „Land = CH?“
- „Farbe $\in$ {rot, orange}?“

Kategorische Merkmale

Echte Datensätze enthalten oft nicht-numerische Merkmale: *Land, Farbe, Blutgruppe*.

Ein Baum verarbeitet sie *direkt*, denn eine Frage kann einfach nach der Kategoriezugehörigkeit fragen:

- „Land = CH?“
- „Farbe $\in$ {rot, orange}?“
- Keine Vorverarbeitung des Merkmals nötig.
- Die Aufteilung bleibt für Menschen lesbar, eine echte praktische Stärke von Bäumen bei tabellarischen Daten mit gemischten Merkmalstypen.

Zusammenfassung

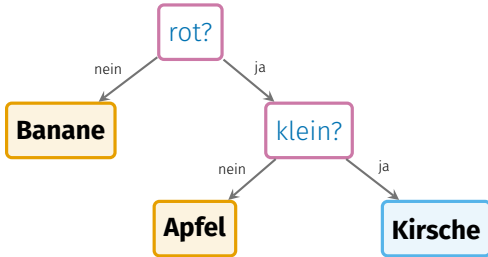
Ein Entscheidungsbaum ist ein Flussdiagramm
aus Ja/Nein-Fragen,
das den Merkmalsraum in achsenparallele
Rechtecke zerlegt.

Gierig wachsen lassen, Unreinheit minimieren;
Tiefe deckeln gegen Auswendiglernen.

2. Theoretische Übungen

Übung 1: Einen Baum durchlaufen

Mit dem Baum unten jede Frucht klassifizieren. Die Merkmale sind *rot?* und *klein?*, beide ja/nein.



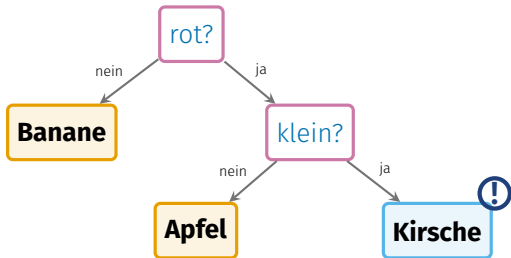
(a) $\mathbf{x}_1 = (\text{rot} = \text{nein}, \text{klein} = \text{ja})$

(b) $\mathbf{x}_2 = (\text{rot} = \text{ja}, \text{klein} = \text{nein})$

(c) $\mathbf{x}_3 = (\text{rot} = \text{ja}, \text{klein} = \text{ja})$

Übung 1: Einen Baum durchlaufen

Mit dem Baum unten jede Frucht klassifizieren. Die Merkmale sind *rot?* und *klein?*, beide ja/nein.



(a) $\mathbf{x}_1 = (\text{rot} = \text{nein}, \text{klein} = \text{ja})$

(b) $\mathbf{x}_2 = (\text{rot} = \text{ja}, \text{klein} = \text{nein})$

(c) $\mathbf{x}_3 = (\text{rot} = \text{ja}, \text{klein} = \text{ja})$



(a) rot? **nein** → **Banane**.

(b) rot? **ja** → klein? **nein** → **Apfel**.

(c) rot? **ja** → klein? **ja** → **Kirsche**.

Übung 2: Einen Gini-Split berechnen

Ein Knoten enthält 6 Datenpunkte: 4 **positive**, 2 **negative**. Eine Kandidaten-Aufteilung $x_1 > 3$ ergibt:

- linkes Kind ($n_L = 3$): 3 **pos**, 0 **neg**
- rechtes Kind ($n_R = 3$): 1 **pos**, 2 **neg**

Erinnerung: $G = 1 - p_+^2 - p_-^2$, mit $p_+ = \frac{\#_{\text{pos}}}{\#_{\text{gesamt}}}$ und $p_- = 1 - p_+$.

Übung 2: Einen Gini-Split berechnen

Ein Knoten enthält 6 Datenpunkte: 4 **positive**, 2 **negative**. Eine Kandidaten-Aufteilung $x_1 > 3$ ergibt:

- linkes Kind ($n_L = 3$): 3 **pos**, 0 **neg**
- rechtes Kind ($n_R = 3$): 1 **pos**, 2 **neg**

Erinnerung: $G = 1 - p_+^2 - p_-^2$, mit $p_+ = \frac{\#_{\text{pos}}}{\#_{\text{gesamt}}}$ und $p_- = 1 - p_+$.

- p_+ , p_- für den **Elternknoten** berechnen und daraus die Gini-Unreinheit G bestimmen.
- G_L und G_R für jedes Kind berechnen. (Was ist G , wenn $p_+ = 1$?)
- $G_{\text{split}} = \frac{n_L}{n} G_L + \frac{n_R}{n} G_R$ berechnen. Ist das eine gute Aufteilung?

Übung 2: Einen Gini-Split berechnen (Lösung)

❗ (a) Elternknoten: $p_+ = 4/6$, $p_- = 2/6$, also

$$G = 1 - (4/6)^2 - (2/6)^2 = 1 - \frac{16}{36} - \frac{4}{36} = \frac{16}{36} \approx 0.44.$$

Übung 2: Einen Gini-Split berechnen (Lösung)

❗ (a) Elternknoten: $p_+ = 4/6$, $p_- = 2/6$, also

$$G = 1 - (4/6)^2 - (2/6)^2 = 1 - \frac{16}{36} - \frac{4}{36} = \frac{16}{36} \approx 0.44.$$

(b) Links ist rein: $G_L = 0$. Rechts: $p_+ = 1/3$, $p_- = 2/3$, also

$$G_R = 1 - (1/3)^2 - (2/3)^2 = 1 - \frac{1}{9} - \frac{4}{9} = \frac{4}{9} \approx 0.44.$$

Übung 2: Einen Gini-Split berechnen (Lösung)

❗ (a) Elternknoten: $p_+ = 4/6$, $p_- = 2/6$, also

$$G = 1 - (4/6)^2 - (2/6)^2 = 1 - \frac{16}{36} - \frac{4}{36} = \frac{16}{36} \approx 0.44.$$

(b) Links ist rein: $G_L = 0$. Rechts: $p_+ = 1/3$, $p_- = 2/3$, also

$$G_R = 1 - (1/3)^2 - (2/3)^2 = 1 - \frac{1}{9} - \frac{4}{9} = \frac{4}{9} \approx 0.44.$$

(c) Gewichtete Kind-Unreinheit:

$$G_{\text{split}} = \frac{3}{6} \cdot 0 + \frac{3}{6} \cdot \frac{4}{9} = \frac{2}{9} \approx 0.22.$$

0.44 $\rightarrow$ 0.22 ist ein deutlicher Rückgang. Ja, das ist eine gute Aufteilung.

Übung 3: Tiefe diagnostizieren

Ein Student trainiert einen Entscheidungsbaum der Tiefe 20 auf $n = 100$ Punkten und berichtet

- Trainingsgenauigkeit = 100%,
- Validierungsgenauigkeit = 60%.

Übung 3: Tiefe diagnostizieren

Ein Student trainiert einen Entscheidungsbaum der Tiefe 20 auf $n = 100$ Punkten und berichtet

- Trainingsgenauigkeit = 100%,
- Validierungsgenauigkeit = 60%.

- (a) Was ist die wahrscheinlichste Diagnose?
- (b) Was sollte als Nächstes probiert werden?

Übung 3: Tiefe diagnostizieren

Ein Student trainiert einen Entscheidungsbaum der Tiefe 20 auf $n = 100$ Punkten und berichtet

- Trainingsgenauigkeit = 100%,
- Validierungsgenauigkeit = 60%.

- (a) Was ist die wahrscheinlichste Diagnose?
- (b) Was sollte als Nächstes probiert werden?

- ❗ (a) Klassische **Überanpassung**: bei Tiefe 20 und nur 100 Punkten schneidet der Baum um jeden Punkt ein winziges Gebiet aus. Jedes Blatt enthält einen Punkt, also 100% Trainingsgenauigkeit. Der Baum lernt Rauschen statt verallgemeinerbarer Struktur.
- (b) Die Tiefe reduzieren: kleinere Tiefen probieren (z. B. 2, 3, ..., 8) und diejenige mit der besten Validierungsgenauigkeit wählen.

3. Programmierübungen

Programmierübungen

Sie haben nun die Kenntnisse, die folgenden Übungen zu lösen:

`https://expert.ethz.ch/enrolled/SS26/mavt2/exercises`

- **Cancer Detection**
- **Gini Index**

4. k-Nearest Neighbors

k-Nearest Neighbors

Die Idee in einem Satz:

k-Nearest Neighbors

Die Idee in einem Satz:

*Um einen neuen Punkt zu klassifizieren, betrachtet man die **k** nächsten Trainingspunkte und lässt sie **abstimmen**.*

k-Nearest Neighbors

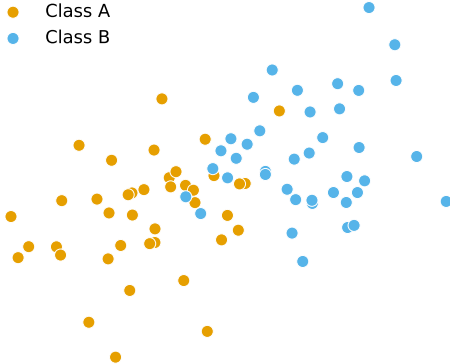
Die Idee in einem Satz:

*Um einen neuen Punkt zu klassifizieren, betrachtet man die **k** nächsten Trainingspunkte und lässt sie **abstimmen**.*

Das ist alles. Es gibt keine Gleichung zu lösen, keine Parameter zu fitten. Das Modell ist schlicht die gesamte Trainingsmenge.

k-Nearest Neighbors: Der Algorithmus

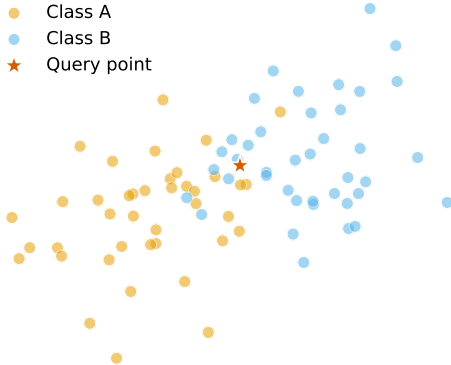
● Class A
● Class B



■ Gelabelte Trainingsmenge mit zwei Klassen (orange, blau).

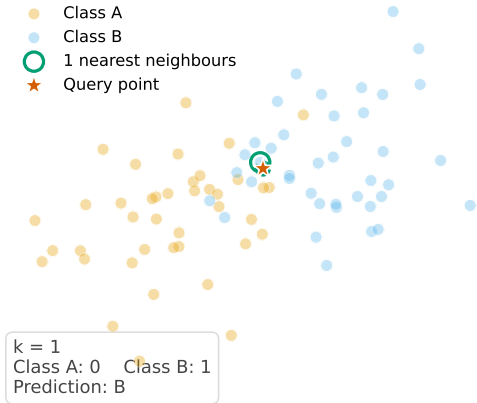
k-Nearest Neighbors: Der Algorithmus

- Class A
- Class B
- ★ Query point



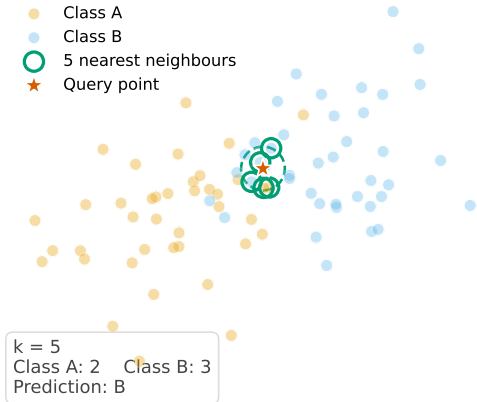
- Gelabelte Trainingsmenge mit zwei Klassen (orange, blau).
- Ein neuer Anfragepunkt: welche Klasse?

k-Nearest Neighbors: Der Algorithmus



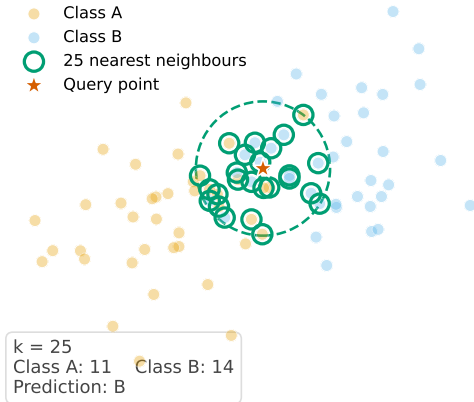
- Gelabelte Trainingsmenge mit zwei Klassen (orange, blau).
- Ein neuer **Anfragepunkt**: welche Klasse?
- $k = 1$: nächster Trainingspunkt entscheidet. Stimme: blau.

k-Nearest Neighbors: Der Algorithmus



- Gelabelte Trainingsmenge mit zwei Klassen (orange, blau).
- Ein neuer **Anfragepunkt**: welche Klasse?
- $k = 1$: nächster Trainingspunkt entscheidet. Stimme: blau.
- $k = 5$: die fünf nächsten. Mehrheitsentscheid.

k-Nearest Neighbors: Der Algorithmus



- Gelabelte Trainingsmenge mit zwei Klassen (orange, blau).
- Ein neuer **Anfragepunkt**: welche Klasse?
- $k = 1$: nächster Trainingspunkt entscheidet. Stimme: blau.
- $k = 5$: die fünf nächsten. Mehrheitsentscheid.
- $k = 25$: fünfundzwanzig. Das Ergebnis kann kippen!

k-Nearest Neighbors: Das Rezept

Gegeben eine Trainingsmenge $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ und ein neuer Anfragepunkt $\mathbf{x}$:

k-Nearest Neighbors: Das Rezept

Gegeben eine Trainingsmenge $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ und ein neuer Anfragepunkt $\mathbf{x}$:

1. Man berechnet die Distanz $d(\mathbf{x}, \mathbf{x}_i)$ zu *jedem* Trainingspunkt.

k-Nearest Neighbors: Das Rezept

Gegeben eine Trainingsmenge $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ und ein neuer Anfragepunkt $\mathbf{x}$:

1. Man berechnet die Distanz $d(\mathbf{x}, \mathbf{x}_i)$ zu *jedem* Trainingspunkt.
2. Man wählt die k Trainingspunkte mit der kleinsten Distanz; diese Menge nennt man $\mathcal{N}_k(\mathbf{x})$.

k-Nearest Neighbors: Das Rezept

Gegeben eine Trainingsmenge $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ und ein neuer Anfragepunkt $\mathbf{x}$:

1. Man berechnet die Distanz $d(\mathbf{x}, \mathbf{x}_i)$ zu *jedem* Trainingspunkt.
2. Man wählt die k Trainingspunkte mit der kleinsten Distanz; diese Menge nennt man $\mathcal{N}_k(\mathbf{x})$.
3. Man gibt das häufigste Label unter ihnen zurück:

$$\hat{y}(\mathbf{x}) = \arg \max_c \sum_{i \in \mathcal{N}_k(\mathbf{x})} \mathbf{1}[y_i = c]$$

k-Nearest Neighbors: Das Rezept

Gegeben eine Trainingsmenge $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ und ein neuer Anfragepunkt $\mathbf{x}$:

1. Man berechnet die Distanz $d(\mathbf{x}, \mathbf{x}_i)$ zu *jedem* Trainingspunkt.
2. Man wählt die k Trainingspunkte mit der kleinsten Distanz; diese Menge nennt man $\mathcal{N}_k(\mathbf{x})$.
3. Man gibt das häufigste Label unter ihnen zurück:

$$\hat{y}(\mathbf{x}) = \arg \max_c \sum_{i \in \mathcal{N}_k(\mathbf{x})} \mathbf{1}[y_i = c]$$

- Für die **Regression**: man gibt den Mittelwert der k Nachbarwerte zurück statt eines Mehrheitsentscheids.

k-Nearest Neighbors: “Lazy Learning”

Man beachte, was im Rezept fehlt: **Es gibt keinen Trainingsschritt.**

k-Nearest Neighbors: “Lazy Learning”

Man beachte, was im Rezept fehlt: **Es gibt keinen Trainingsschritt.**

- Parametrische Modelle wie lineare oder logistische Regression (wie später noch zu sehen): Training ist teuer, Vorhersage ist günstig (ein Skalarprodukt).

k-Nearest Neighbors: “Lazy Learning”

Man beachte, was im Rezept fehlt: **Es gibt keinen Trainingsschritt.**

- Parametrische Modelle wie lineare oder logistische Regression (wie später noch zu sehen): Training ist teuer, Vorhersage ist günstig (ein Skalarprodukt).
- k-NN: Training ist *gratis* (nur die Daten speichern), aber *jede* Vorhersage kostet $O(n \cdot D)$, weil man den ganzen Datensatz durchgehen muss.

k-Nearest Neighbors: “Lazy Learning”

Man beachte, was im Rezept fehlt: **Es gibt keinen Trainingsschritt.**

- Parametrische Modelle wie lineare oder logistische Regression (wie später noch zu sehen): Training ist teuer, Vorhersage ist günstig (ein Skalarprodukt).
- k-NN: Training ist *gratis* (nur die Daten speichern), aber *jede* Vorhersage kostet $O(n \cdot D)$, weil man den ganzen Datensatz durchgehen muss.
- k-NN nennt man einen **lazy** oder **nicht-parametrischen** Lerner: er verschiebt die ganze Arbeit auf den Moment der Anfrage.

k-Nearest Neighbors: Welcher Abstand?

“Am nächsten” hängt davon ab, wie man die Distanz misst. Standardmässig verwendet man die **euklidische Distanz**:

$$d(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_{j=1}^D (x_j - x'_j)^2}$$

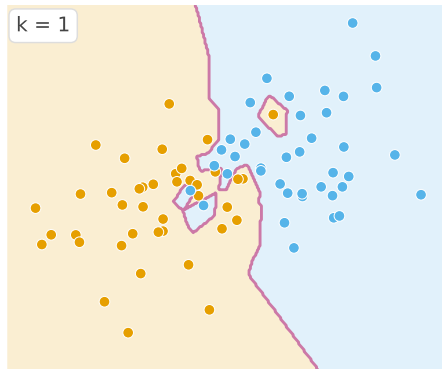
k-Nearest Neighbors: Welcher Abstand?

“Am nächsten” hängt davon ab, wie man die Distanz misst. Standardmässig verwendet man die **euklidische Distanz**:

$$d(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_{j=1}^D (x_j - x'_j)^2}$$

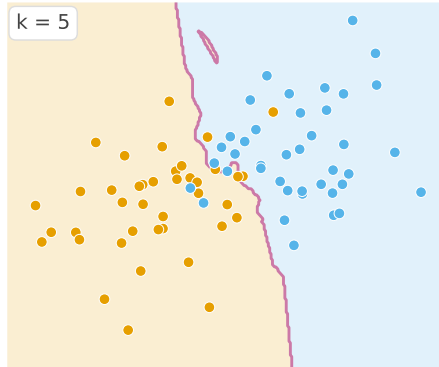
Andere Wahlmöglichkeiten existieren, aber für diesen Kurs genügt es, die euklidische Distanz im Kopf zu haben. Man erinnere sich daran, dass die euklidische Distanz nichts anderes ist als die Länge der geraden Linie, die zwei Punkte im Raum verbindet.

Wirkung von k : Entscheidungsgrenze



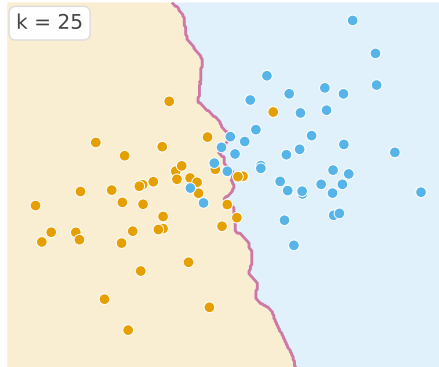
$k = 1$: die Entscheidungsgrenze legt sich um jeden Trainingspunkt, ist zackig und folgt dem Rauschen.

Wirkung von k : Entscheidungsgrenze



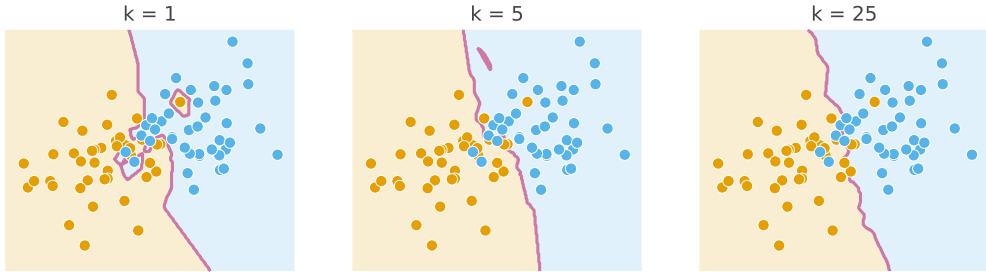
$k = 5$: glatter, ignoriert isolierte Ausreisser.

Wirkung von k : Entscheidungsgrenze



$k = 25$: sehr glatt, fast eine Gerade, verliert aber feine Details.

Wirkung von k : Entscheidungsgrenze



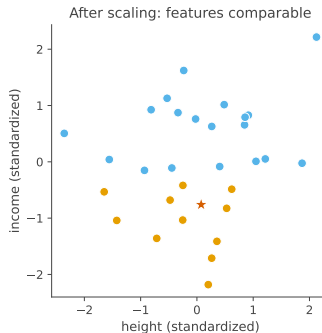
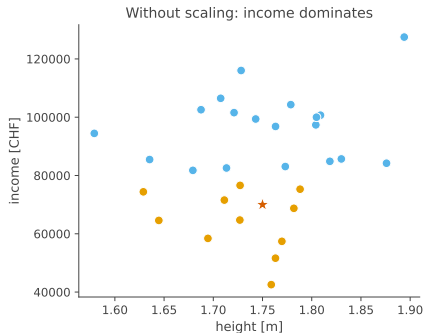
Im Vergleich: kleines k overfittet, grosses k underfittet (mehr dazu in der Trainings-Lektion).

Vorsicht: Skalierung der Merkmale

k-NN nutzt rohe Distanzen: ein Merkmal mit grossem Wertebereich dominiert.

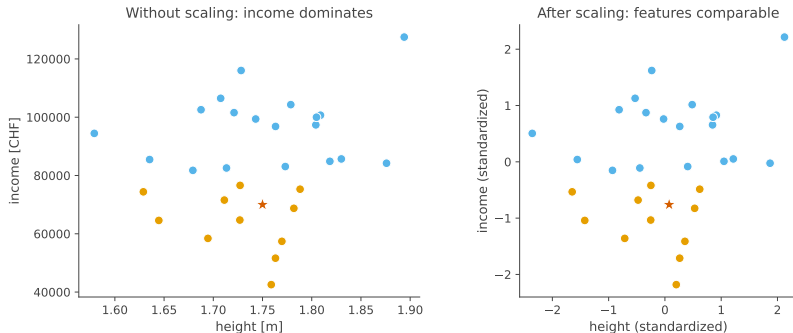
Vorsicht: Skalierung der Merkmale

k-NN nutzt rohe Distanzen: ein Merkmal mit grossem Wertebereich dominiert.



Vorsicht: Skalierung der Merkmale

k-NN nutzt rohe Distanzen: ein Merkmal mit grossem Wertebereich dominiert.



→ Vor dem Anwenden von k-NN die Merkmale stets **standardisieren**.

Zusammenfassung

Stärken

- Konzeptuell trivial.
- Keine Trainingskosten.
- Erfasst nicht-lineare Grenzen ohne Aufwand.
- Funktioniert für Klassifikation *und* Regression.

Schwächen

- Langsam zur Vorhersagezeit: $O(nD)$ pro Anfrage.
- Speichert den ganzen Datensatz.
- Empfindlich gegenüber Merkmalsskalierung.
- Leidet in hohen Dimensionen ("Fluch der Dimensionalität").

Zusammenfassung

Stärken

- Konzeptuell trivial.
- Keine Trainingskosten.
- Erfasst nicht-lineare Grenzen ohne Aufwand.
- Funktioniert für Klassifikation *und* Regression.

Schwächen

- Langsam zur Vorhersagezeit: $O(nD)$ pro Anfrage.
- Speichert den ganzen Datensatz.
- Empfindlich gegenüber Merkmalsskalierung.
- Leidet in hohen Dimensionen (“Fluch der Dimensionalität”).

*k ist der einzige Stellknopf, ein **Hyperparameter**, wie in der Trainings-Lektion zu sehen sein wird*

5. Theoretische Übungen

Übung 1: Klassifikation von Hand

Gegeben sind die folgenden gelabelten Trainingspunkte in $\mathbb{R}^2$:

i	x_1	x_2	y
1	1	1	A
2	2	1	A
3	4	3	B
4	5	4	B
5	3	4	B

Man klassifiziere den Anfragepunkt $\mathbf{x} = (3, 2)$ mit **(a)** $k = 1$ und **(b)** $k = 3$.

Übung 1: Klassifikation von Hand (Lösung)

Distanzen von $\mathbf{x} = (3, 2)$:

i	$d(\mathbf{x}, \mathbf{x}_i)$	y_i
1	$\sqrt{(3-1)^2 + (2-1)^2} = \sqrt{5} \approx 2.24$	A
2	$\sqrt{1^2 + 1^2} = \sqrt{2} \approx 1.41$	A
3	$\sqrt{1^2 + 1^2} = \sqrt{2} \approx 1.41$	B
4	$\sqrt{2^2 + 2^2} = \sqrt{8} \approx 2.83$	B
5	$\sqrt{0^2 + 2^2} = 2$	B

Übung 1: Klassifikation von Hand (Lösung)

Distanzen von $\mathbf{x} = (3, 2)$:

i	$d(\mathbf{x}, \mathbf{x}_i)$	y_i
1	$\sqrt{(3-1)^2 + (2-1)^2} = \sqrt{5} \approx 2.24$	A
2	$\sqrt{1^2 + 1^2} = \sqrt{2} \approx 1.41$	A
3	$\sqrt{1^2 + 1^2} = \sqrt{2} \approx 1.41$	B
4	$\sqrt{2^2 + 2^2} = \sqrt{8} \approx 2.83$	B
5	$\sqrt{0^2 + 2^2} = 2$	B

- ❗ (a) $k = 1$: Stichentscheid zwischen Punkt 2 (A) und Punkt 3 (B), beide mit Distanz $\sqrt{2}$. A oder B ist vertretbar; ein typischer Stichentscheid ist, den nächstfolgenden Nachbarn heranzuziehen.
- (b) $k = 3$: die drei nächsten sind 2, 3, 5 mit Labels A, B, B. Mehrheit: **B**.

Übung 2: k wählen

Man entscheide für jede Aussage, ob sie **wahr** oder **falsch** ist, und begründe kurz.

- (c) Wenn k gleich der Grösse der Trainingsmenge ist, ist die Vorhersage für jeden Anfragepunkt dieselbe.
- (d) k -NN kann eine nicht-lineare Entscheidungsgrenze lernen, obwohl es keine lernbaren Parameter besitzt.

Übung 2: k wählen

Man entscheide für jede Aussage, ob sie **wahr** oder **falsch** ist, und begründe kurz.

- (c) Wenn k gleich der Grösse der Trainingsmenge ist, ist die Vorhersage für jeden Anfragepunkt dieselbe.
- (d) k -NN kann eine nicht-lineare Entscheidungsgrenze lernen, obwohl es keine lernbaren Parameter besitzt.
- (c) **Wahr.** Die Vorhersage ist die Mehrheitsklasse der gesamten Trainingsmenge.
- (d) **Wahr.** Mit genügend Daten kann die Grenze jede Form annehmen.

Übung 3: Eine Falle bei der Merkmalsskalierung

Man baut einen k-NN-Klassifikator, um Fahrräder zu empfehlen. Jede Kundin und jeder Kunde wird durch *Körpergrösse* (Meter, Bereich ~ 1.5 – 2.0) und *Jahreseinkommen* (CHF, Bereich $\sim 30,000$ – $200,000$) beschrieben.

Ohne jegliche Vorverarbeitung trainiert man k-NN mit $k = 5$ und stellt fest, dass das Modell die Körpergrösse vollständig ignoriert.

Übung 3: Eine Falle bei der Merkmalsskalierung

Man baut einen k-NN-Klassifikator, um Fahrräder zu empfehlen. Jede Kundin und jeder Kunde wird durch *Körpergrösse* (Meter, Bereich ~ 1.5 – 2.0) und *Jahreseinkommen* (CHF, Bereich $\sim 30,000$ – $200,000$) beschrieben.

Ohne jegliche Vorverarbeitung trainiert man k-NN mit $k = 5$ und stellt fest, dass das Modell die Körpergrösse vollständig ignoriert.

(a) Wieso passiert das?

(b) Welcher einzige Vorverarbeitungsschritt behebt das?

Übung 3: Eine Falle bei der Merkmalsskalierung

Man baut einen k-NN-Klassifikator, um Fahrräder zu empfehlen. Jede Kundin und jeder Kunde wird durch *Körpergrösse* (Meter, Bereich $\sim 1.5\text{--}2.0$) und *Jahreseinkommen* (CHF, Bereich $\sim 30,000\text{--}200,000$) beschrieben.

Ohne jegliche Vorverarbeitung trainiert man k-NN mit $k = 5$ und stellt fest, dass das Modell die Körpergrösse vollständig ignoriert.

(a) Wieso passiert das?

(b) Welcher einzige Vorverarbeitungsschritt behebt das?

❗ (a) Die Distanzen werden vom Einkommen dominiert: eine Differenz von CHF 10,000 trägt $10,000^2$ zur quadrierten Distanz bei, jede Differenz in der Körpergrösse weniger als 1. Die Körpergrösse ist unsichtbar.

(b) Die Merkmale **standardisieren**, damit sie in vergleichbaren Bereichen liegen.

6. Lineare Regression

Was ist Regression?

- **Regression:** aus Merkmalen einen kontinuierlichen Wert vorhersagen.

Was ist Regression?

- **Regression:** aus Merkmalen einen kontinuierlichen Wert vorhersagen.
- **Lineare Regression:** die Vorhersage erfolgt über eine lineare Abbildung.

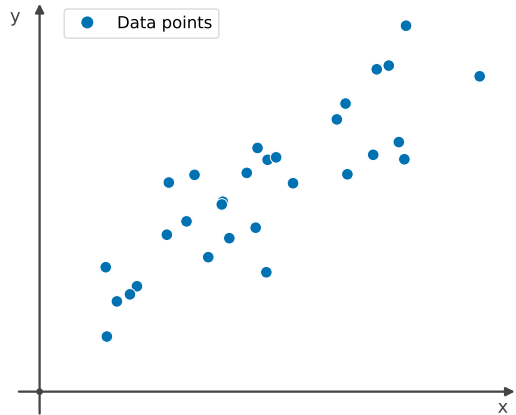
Was ist Regression?

- **Regression:** aus Merkmalen einen kontinuierlichen Wert vorhersagen.
- **Lineare Regression:** die Vorhersage erfolgt über eine lineare Abbildung.
- Geometrisch: eine Gerade (oder Hyperebene) durch die Daten legen.

Der Datensatz

Merkmale $\mathbf{x}_i$		Label y_i
Lern h	Schlaf h	Punkte
2	7	58
5	6	74
8	8	91
$\vdots$	$\vdots$	$\vdots$

$$\mathbf{x}_i \in \mathbb{R}^D, \quad y_i \in \mathbb{R}, \quad i = 1, \dots, n.$$



ein Merkmal ($D = 1$) zur Visualisierung dargestellt

Das Modell

$$\hat{y} = w_0 + w_1 x$$

Ein Merkmal: eine Gerade mit Steigung w_1 und Achsenabschnitt w_0 .

Das Modell

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \dots + w_D x_D$$

D Merkmale: jedes w_j ist das Gewicht des Merkmals x_j .

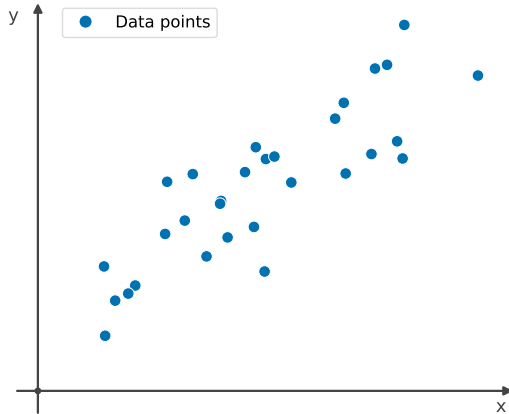
Das Modell

$$\hat{y} = \mathbf{w} \cdot \mathbf{x}$$

Kompakte Form: man stellt der $\mathbf{x}$ eine 1 voran, damit der Bias w_0 mit ihr multipliziert wird.

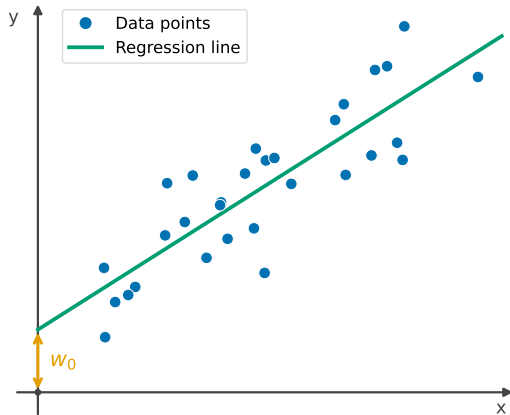
$$\mathbf{w} = (w_0, w_1, \dots, w_D), \quad \mathbf{x} = (1, x_1, \dots, x_D).$$

Beispiel: $n = 30$, $D = 1$



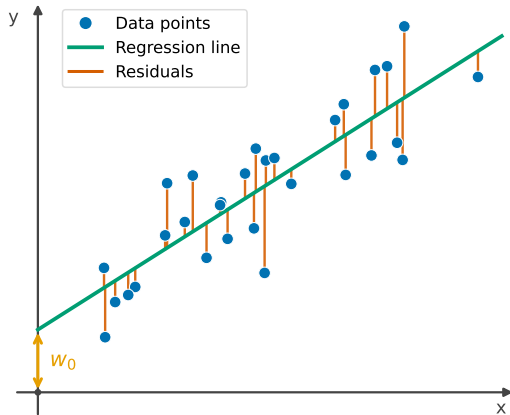
- Die Daten: ein Merkmal auf der x -Achse, das Label auf der y -Achse.

Beispiel: $n = 30$, $D = 1$



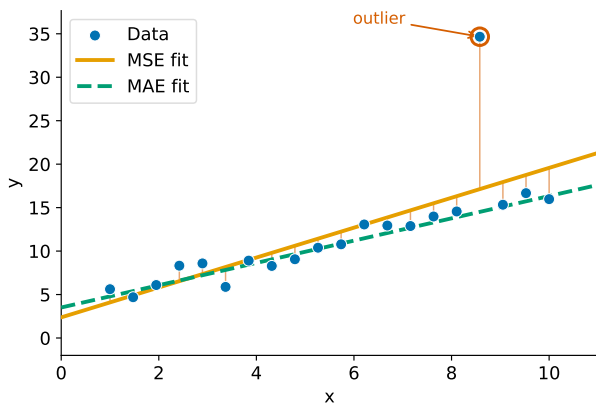
- Die Daten: ein Merkmal auf der x -Achse, das Label auf der y -Achse.
- Die Anpassung. w_0 ist der y -Achsenabschnitt.

Beispiel: $n = 30, D = 1$



- Die Daten: ein Merkmal auf der x -Achse, das Label auf der y -Achse.
- Die Anpassung. w_0 ist der y -Achsenabschnitt.
- Die Residuen $\hat{y}_i - y_i$, genau das, was der Verlust misst.

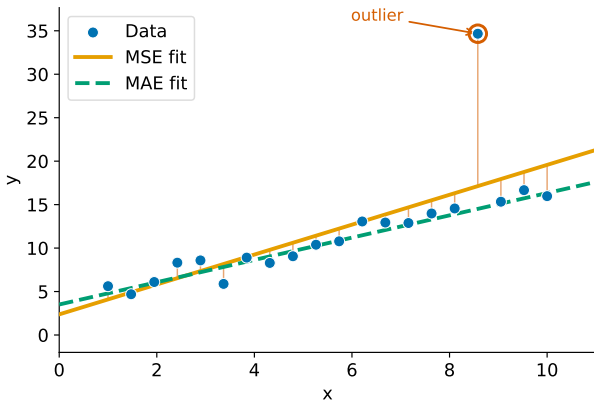
Verlustfunktion



$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

Verlustfunktion



$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

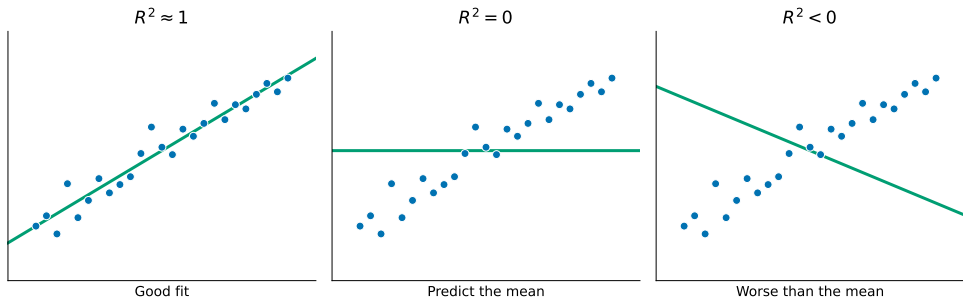
$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

MSE quadriert die Fehler, sodass ein einzelner **Ausreisser** die Gerade an sich zieht. MAE ist robust. Allerdings behandelt der MAE alle Fehler gleich, wodurch grosse Abweichungen, die in manchen Anwendungen von Bedeutung sind, möglicherweise untergewichtet werden.

Die R^2 -Kennzahl

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

vergleicht das Modell mit der Mittelwert-Basislinie



Anders als der Verlust ist R^2 einheitslos und zwischen Datensätzen vergleichbar.

Linear in den Gewichten

„Linear“ bezieht sich auf die **Gewichte**, *nicht* auf die Eingabe.

Linear in den Gewichten

„Linear“ bezieht sich auf die Gewichte, *nicht* auf die Eingabe.

Gekrümmte Daten? Einfach neue Merkmale hinzufügen:

$$\hat{y} = w_0 + w_1 x + w_2 x^2$$

weiterhin lineare Regression, nur mit $\mathbf{x} = (1, x, x^2)$.

Linear in den Gewichten

„Linear“ bezieht sich auf die **Gewichte**, *nicht* auf die Eingabe.

Gekrümmte Daten? Einfach neue Merkmale hinzufügen:

$$\hat{y} = w_0 + w_1 x + w_2 x^2$$

weiterhin lineare Regression, nur mit $\mathbf{x} = (1, x, x^2)$.

Derselbe Trick passt Parabeln (geworfener Ball), v^2 (Bremsweg), $\log x$, $\sin x$, ...

Zusammenfassung

Lineare Regression passt eine Hyperebene an, indem sie den quadratischen Fehler minimiert.

„Linear“ bezieht sich auf die Gewichte,
nicht auf die Merkmale.

7. Theoretische Übungen

Übung 1: Wahl der Loss-Funktion

Gegeben sei eine lineare Regression zur Vorhersage von Hauspreisen. Ein Datenpunkt zeigt einen Preis von \$10,000,000, vermutlich ein Tippfehler: er sollte wohl \$1,000,000 lauten.

Zur Wahl stehen zwei Verlustfunktionen:

- Mittlerer quadratischer Fehler (MSE): $\frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$
- Mittlerer absoluter Fehler (MAE): $\frac{1}{n} \sum_i |\hat{y}_i - y_i|$

Übung 1: Wahl der Loss-Funktion

Gegeben sei eine lineare Regression zur Vorhersage von Hauspreisen. Ein Datenpunkt zeigt einen Preis von \$10,000,000, vermutlich ein Tippfehler: er sollte wohl \$1,000,000 lauten.

Zur Wahl stehen zwei Verlustfunktionen:

- Mittlerer quadratischer Fehler (MSE): $\frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$
- Mittlerer absoluter Fehler (MAE): $\frac{1}{n} \sum_i |\hat{y}_i - y_i|$

Welche ist vorzuziehen, und warum?

Übung 1: Wahl der Loss-Funktion

Gegeben sei eine lineare Regression zur Vorhersage von Hauspreisen. Ein Datenpunkt zeigt einen Preis von \$10,000,000, vermutlich ein Tippfehler: er sollte wohl \$1,000,000 lauten.

Zur Wahl stehen zwei Verlustfunktionen:

- Mittlerer quadratischer Fehler (MSE): $\frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$
- Mittlerer absoluter Fehler (MAE): $\frac{1}{n} \sum_i |\hat{y}_i - y_i|$

Welche ist vorzuziehen, und warum?

❗ **MAE.** Der MSE quadriert den Fehler dieses einen Ausreissers, sodass ein einzelner weit entfernter Punkt die gesamte Anpassung zu sich ziehen kann. Der MAE bestraft Fehler linear und ist daher deutlich robuster gegenüber Ausreissern.

Übung 2: Gewichte interpretieren

Gegeben sei eine lineare Regression zur Vorhersage der Prüfungspunkte (von 100). Alle Merkmale sind ähnlich skaliert, das Modell lautet:

$$\hat{y} = 50 + 5 \cdot \text{Lernstd.} - 3 \cdot \text{Handystd.} + 2 \cdot \text{Schlafstd.}$$

Übung 2: Gewichte interpretieren

Gegeben sei eine lineare Regression zur Vorhersage der Prüfungspunkte (von 100). Alle Merkmale sind ähnlich skaliert, das Modell lautet:

$$\hat{y} = 50 + 5 \cdot \text{Lernstd.} - 3 \cdot \text{Handystd.} + 2 \cdot \text{Schlafstd.}$$

- (a) Welches Merkmal hat den stärksten *negativen* Einfluss?
- (b) Eine Stunde mehr Lernen (sonst gleich): wie ändert sich die Vorhersage?
- (c) Was bedeutet die Konstante 50?

Übung 2: Gewichte interpretieren

Gegeben sei eine lineare Regression zur Vorhersage der Prüfungspunkte (von 100). Alle Merkmale sind ähnlich skaliert, das Modell lautet:

$$\hat{y} = 50 + 5 \cdot \text{Lernstd.} - 3 \cdot \text{Handystd.} + 2 \cdot \text{Schlafstd.}$$

- (a) Welches Merkmal hat den stärksten *negativen* Einfluss?
- (b) Eine Stunde mehr Lernen (sonst gleich): wie ändert sich die Vorhersage?
- (c) Was bedeutet die Konstante 50?

- ❗ (a) Handystd., mit Gewicht -3 .
- (b) Die Vorhersage steigt um 5 Punkte.
- (c) Der Bias w_0 : die Vorhersage bei Wert 0 für alle Merkmale.

Übung 3: Lineare Regression für nichtlineare Daten

Gegeben seien Messungen des Bremswegs eines Autos bei verschiedenen Geschwindigkeiten. Aufgetragen, zeigen die Daten eine deutlich aufwärts gekrümmte Kurve; eine Gerade passt schlecht.

- (a) Liefert eine lineare Anpassung $\hat{y} = w_0 + w_1 \cdot v$ gute Resultate?
- (b) Kann lineare Regression die Daten dennoch modellieren? Falls ja, wie?

Übung 3: Lineare Regression für nichtlineare Daten

Gegeben seien Messungen des Bremswegs eines Autos bei verschiedenen Geschwindigkeiten. Aufgetragen, zeigen die Daten eine deutlich aufwärts gekrümmte Kurve; eine Gerade passt schlecht.

- (a) Liefert eine lineare Anpassung $\hat{y} = w_0 + w_1 \cdot v$ gute Resultate?
- (b) Kann lineare Regression die Daten dennoch modellieren? Falls ja, wie?

❗ (a) Nein, eine Gerade erfasst die Krümmung nicht.

(b) Ja. Man nimmt v^2 als zusätzliches Merkmal: $\hat{y} = w_0 + w_1 v + w_2 v^2$. Linear in den Gewichten w_0, w_1, w_2 , obwohl quadratisch in v . Physikalisch passt das: der Bremsweg ist proportional zur kinetischen Energie $\propto v^2$.

8. Programmierübung

Sie haben nun die Kenntnisse, die folgenden Übungen zu lösen:

<https://expert.ethz.ch/enrolled/SS26/mavt2/exercises>

■ **Diabetes Prediction**